Davante

Desarrollo de Aplicaciones Multiplataforma

Programación Multimedia y Dispositivos Móviles

Tema 5. Programación en Android III: botones y listas

Plantilla de entrega

Tarea 5. Plantilla de entrega

Tema 5. Programación en Android III: botones y listas · Rellena esta plantilla a medida que avanzas, expórtala a PDF y entrégala como PMDM_T05_Apellidos_Nombre.pdf.

Tus datos

Nombre y apellidos	Norberto Guijosa López
URL de la carpeta tema05 en GitHub	*https://github.com/norbeguijosa07-design/PMDM/tree/main/tema05*
Tema de tu lista (ejercicio 2)	*Videojuegos*

Antes de empezar

- Dos proyectos, «Login» y «Lista» seguidos de tu nombre, los dos con **Empty Views Activity** y **Java**. En el asistente sale Kotlin por defecto: cámbialo antes de pulsar Finish.
- Haz un commit al terminar cada apartado del ejercicio 1 y cada paso del ejercicio 2, y copia aquí sus mensajes.
- Tu repositorio tiene que ser **público**. Compruébalo abriendo la URL en una ventana de incógnito.
- Las capturas tienen que ser tuyas y hechas en tu equipo. Cuida la presentación: capturas recortadas a lo que importa y legibles, y textos sin faltas. La calidad de la entrega cuenta en la nota.
- Sustituye el texto gris de cada recuadro por lo que se pide. No borres los títulos.

Ejercicio 1. Pantalla de acceso

Pistas

- **Punto de partida:** el vídeo «Pantalla de login» del campus monta el diseño y la comprobación. Allí se usa `Switch` y los textos van en el código; en tu tarea, MaterialSwitch y `strings.xml`.
- **MaterialSwitch:** arrastra un Switch desde la paleta (Buttons) y, en el XML, cambia `<Switch` por `<com.google.android.material.materialswitch.MaterialSwitch`.
- **Leer lo escrito:** `etCorreo.getText().toString().trim()`. El usuario válido, en dos constantes de la clase.
- **¿Está activado el interruptor?** `swRecordar.isChecked()` devuelve true o false.

Constantes, al principio de MainActivity

```
private static final String CORREO_VALIDO = "diego@correo.com";
private static final String CLAVE_VALIDA = "1234";
```

El botón, en onCreate

```
btnContinuar.setOnClickListener(v -> {
    String correo = etCorreo.getText().toString().trim();
    String clave = etClave.getText().toString();

    if (correo.equals(CORREO_VALIDO) && clave.equals(CLAVE_VALIDA)) {
        // correcto, en verde (¿y si swRecordar está activado?)
    } else {
        // incorrecto, en rojo
    }
});
```

Texto y color del mensaje, con tus recursos

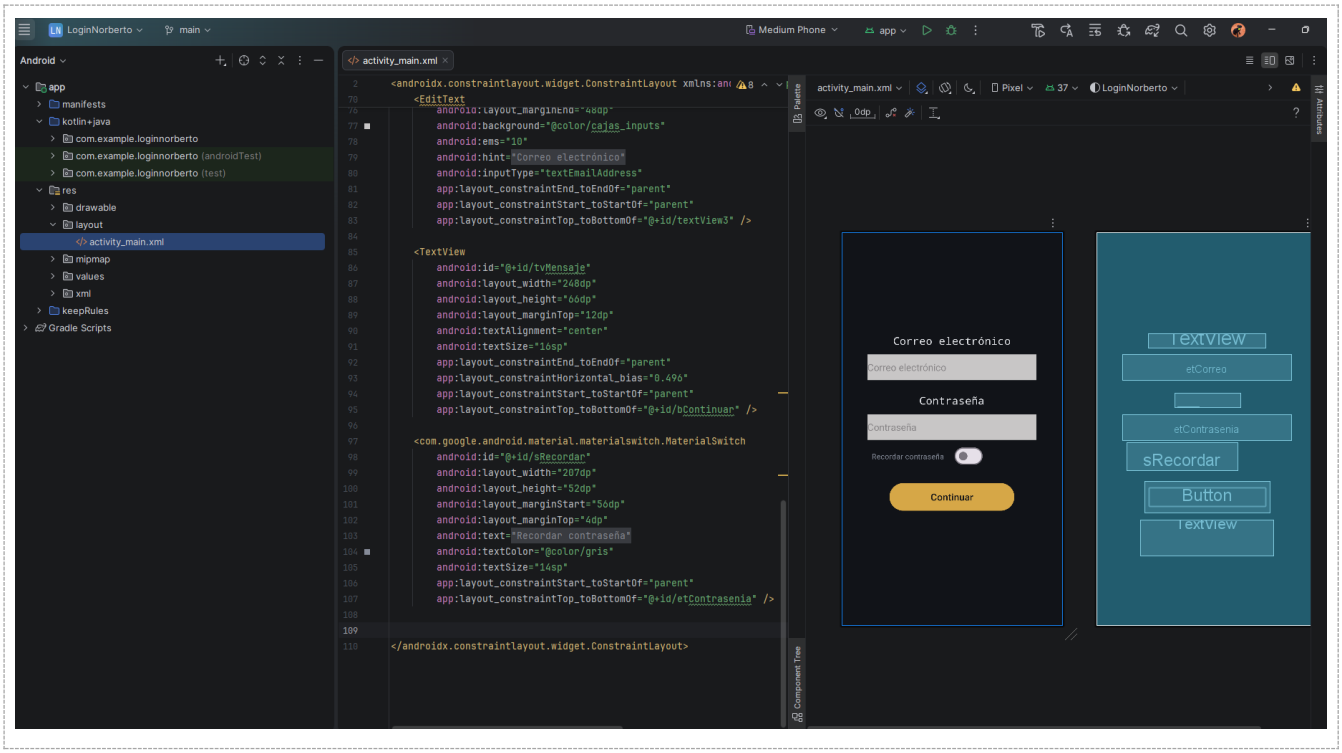
```
tvMensaje.setText(R.string.acceso_correcto);
tvMensaje.setTextColor(ContextCompat.getColor(this, R.color.correcto));
```

1.4 · Campo vacío: avisa y sal del lambda sin comprobar nada más

```
if (correo.isEmpty()) {
    etCorreo.setError(getString(R.string.falta_correo));
    return;
}
```

Extra: el contador de intentos tiene que ser un atributo de la actividad (`private int intentos = 0;`), no una variable dentro del lambda. Para desactivar el botón: `btnContinuar.setEnabled(false)`.

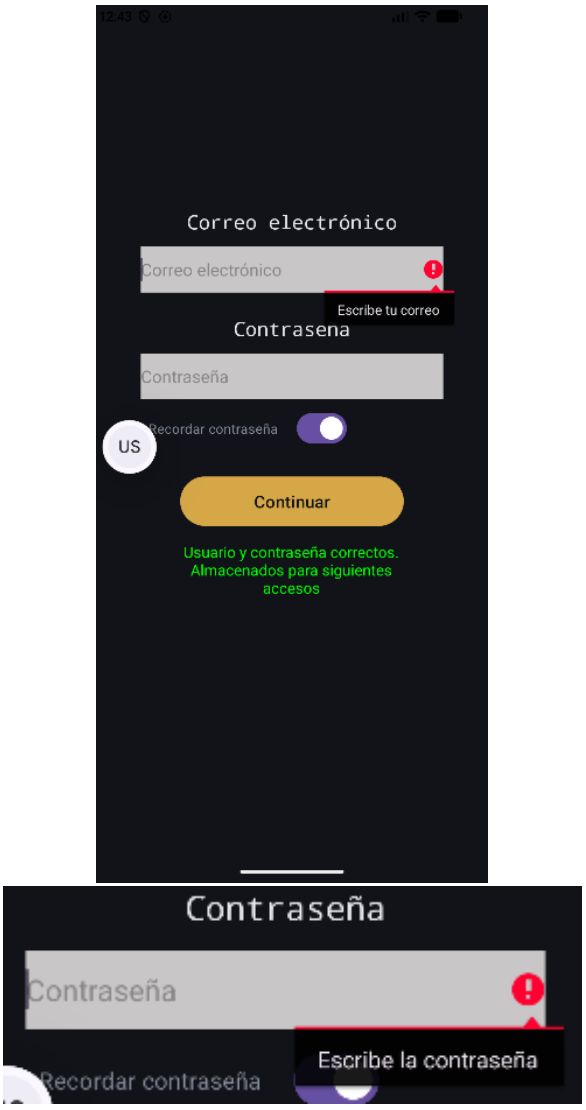
1.a. Captura en modo Split de `activity_main.xml`, con el nombre del proyecto a la vista.



1.b – 1.d. La app en el emulador con los tres mensajes.

1.b. Correcto	1.c. Incorrecto	1.d. Correcto con «Recordar»
		

1.e. El aviso de un campo vacío y, si haces el extra, el botón desactivado tras tres intentos.

1.e. Campo vacío (setError)	Extra. Tres intentos
	

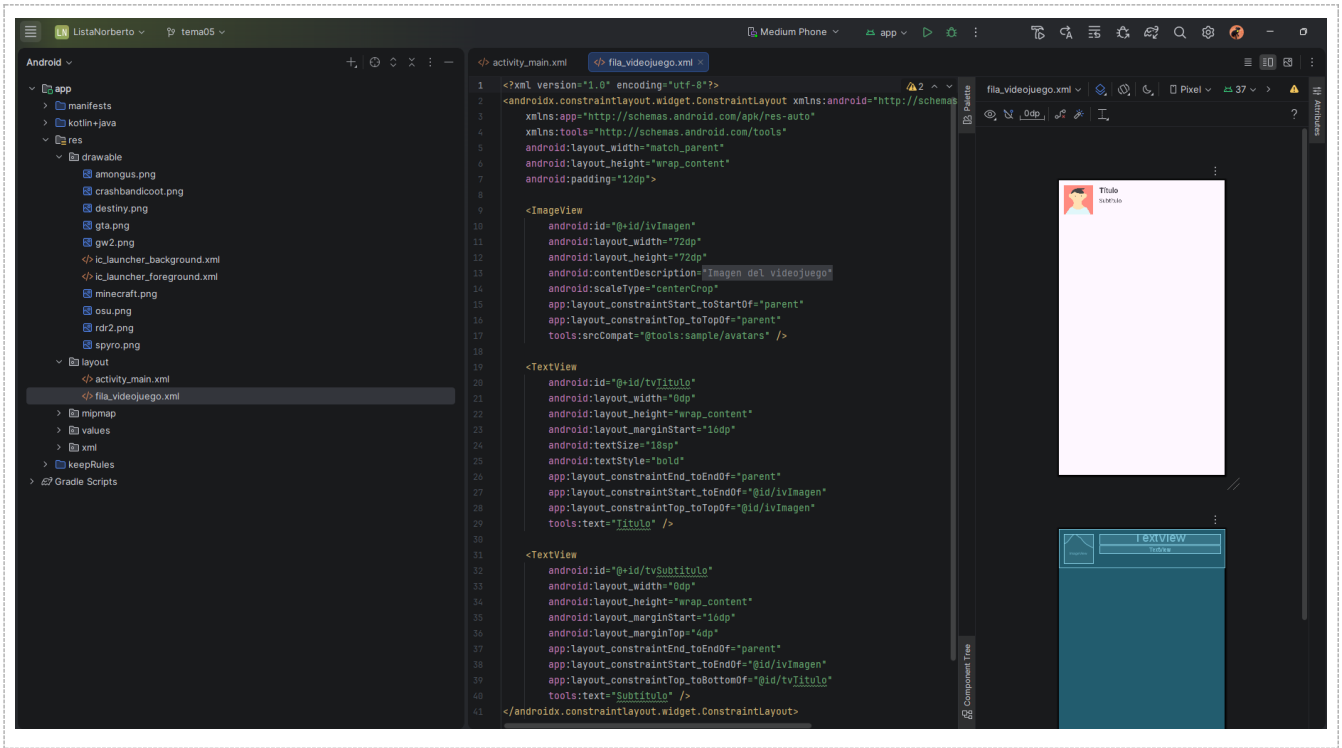
Commit 1.1	1.1: Diseño del Layout de la pantalla de acceso
Commit 1.2 y 1.3	1.2 y 1.3: Comprobación del acceso y recordar
Commit 1.4 (y extra)	1.4: Aviso de campos vacíos 1.5: (Punto Extra) agregar contador de intentos con un máximo de tres intentos que se reinicia al usar los datos correctos

Ejercicio 2. Lista optimizada

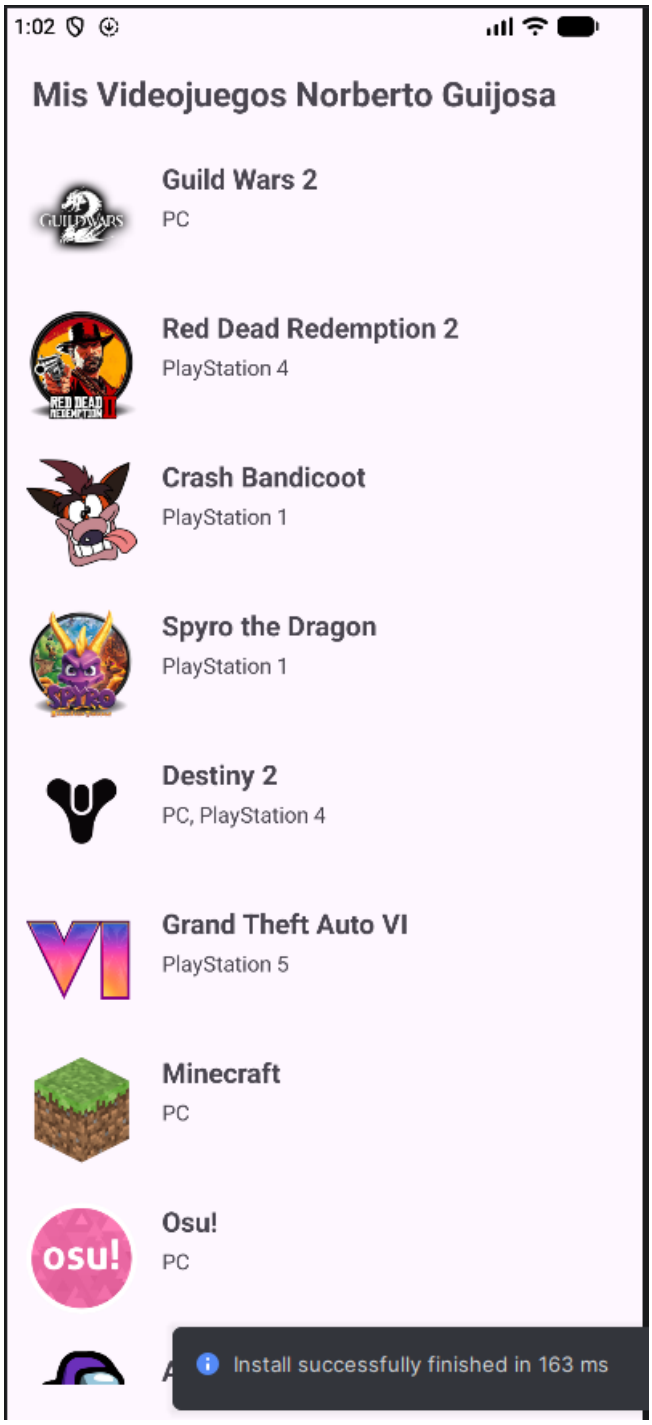
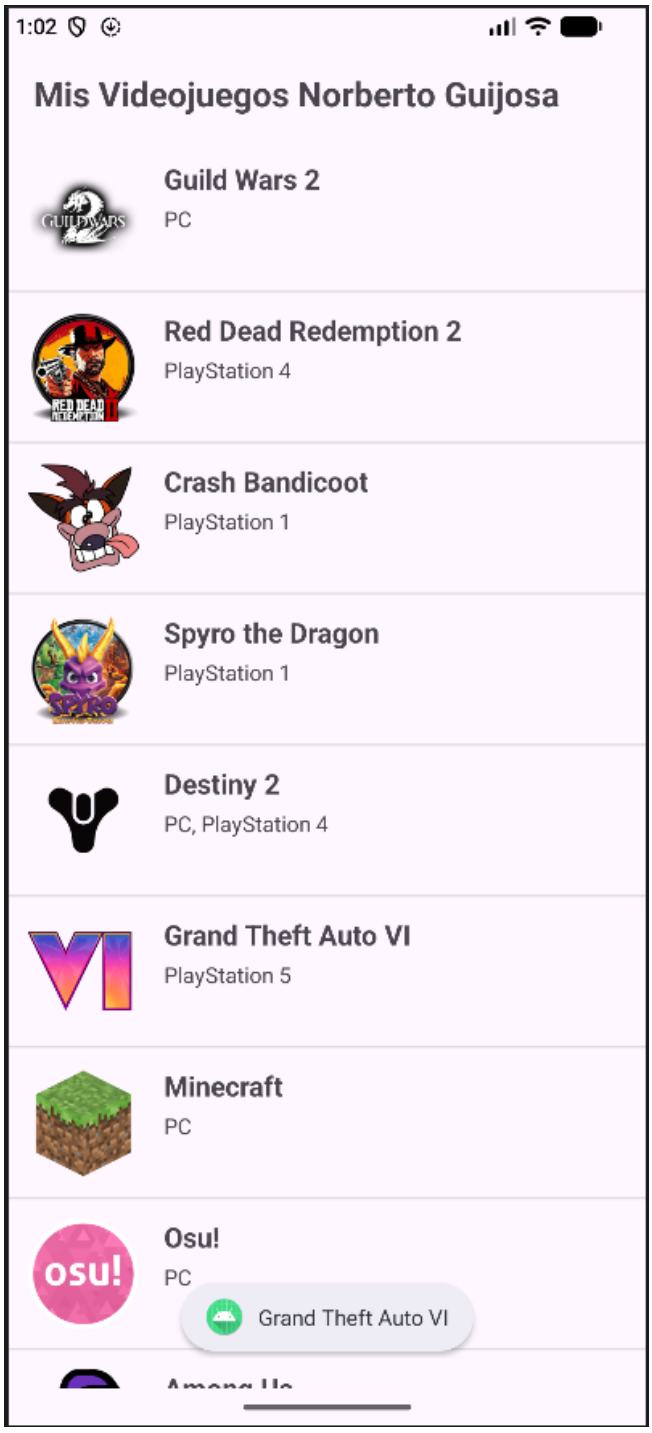
Sigue la guía del enunciado paso a paso. Aquí van las evidencias.

Paso 1 · Proyecto e imágenes	Paso 1: Proyecto e imágenes
Paso 2 · Clase del elemento	Paso 2: clase del elemento VideoJuegos.java
Paso 3 · Layout de la fila	Paso 3: Layout de la fila
Paso 4 · Adaptador	Paso 4: Creación de adaptador AdaptadorVideoJuegos.java
Paso 5 · RecyclerView	Paso 5: RecyclerView en activity_main
Paso 6 · Datos y lista	Paso 6: Datos y lista en mainActivity
Paso 7 · Separadores y clic	Paso 7: Separadores y clics

2.a. Captura en modo Split de tu layout de fila (paso 3).



2.b y 2.c. La app en el emulador.

2.b. Tu lista con tu cabecera (paso 6)	2.c. Separadores y Toast (paso 7)
	

Si algo falla

- **La app se cierra al abrirse:** has borrado el ConstraintLayout de `activity_main.xml` o su id `main`, o en `onCreateViewHolder` pusiste `true` en lugar de `false`.
- **La pantalla sale vacía:** falta `setLayoutManager` o `setAdapter`.
- **Cada fila ocupa toda la pantalla:** la raíz de tu layout de fila tiene que ir a `wrap_content`.
- **El adaptador sale en rojo:** Alt+Intro → Implement methods.
- **Una clase sale en rojo:** Alt+Intro → Import class.
- **R.drawable.algo en rojo:** el nombre no coincide con el de la imagen (minúsculas, sin espacios ni tildes).

Historial de commits

Captura del historial de commits de tu repositorio en GitHub, donde se vean los mensajes de los dos ejercicios.



Un problema que encontraste

No he tenido problemas a la hora de desarrollar la actividad, más allá de una confusión, ya que usé un switch normal en vez de un MaterialSwitch. Por ese motivo, al final del trabajo tuve que hacer algunos pequeños arreglos. También, a la hora de crear la lista, intenté escribir el código a mano para entender lo que hacía, pero cometí algunos errores que corregí cuando me di cuenta.

Uso de IA

He usado IA en este trabajo, pero la verdad es que no ha sido muy necesaria, ya que todo venía explicado por el profesor. He usado Claude para saber cuál era la sintaxis exacta para hacer aparecer el Toast sin tener que buscarla en los apuntes y para ver si podía modificar de forma sencilla el color de los botones de la lista al ser presionados, aunque al final solo he utilizado lo del Toast.